

Compiler Design — Question & Answers

Code Optimization, DAG, Data-Flow Analysis

Q1. What is DAG? Construct the DAG for the given statements, find legal evaluation orders and node names.

Definition of DAG

A **Directed Acyclic Graph (DAG)** is a data structure used to represent the structure of basic-block computations. Each node in the DAG represents a value or an intermediate computation. Leaves represent identifiers (variables) or constants, while interior nodes represent operators. Unlike expression trees, a DAG *shares* identical sub-expressions rather than duplicating them, which is the basis for detecting **common sub-expression elimination (CSE)**.

Key properties:

- Directed edges go from operator node to operand nodes (children).
- No cycles — hence Acyclic.
- Identical sub-expressions map to the *same* node.

Given Three-Address Instructions (in order of execution)

Instruction	Operation
t8 = d + e	Node for 'd+e'
t6 = a + b	Node for 'a+b'
t5 = t6 - c	Node for 't6-c' (reuses t6)
t4 = t5 * t8	Node for 't5*t8'
t3 = t4 - e	Node for 't4-e'
t2 = t6 + t4	Node for 't6+t4' (reuses t6, t4)
t1 = t2 * t3	Node for 't2*t3'

DAG Node Structure (textual representation)

Each node is labeled with its operator; attached variable names are shown in [brackets].

Node 1 (LEAF): a

Node 2 (LEAF): b

Node 3 (LEAF): c

Node 4 (LEAF): d

Node 5 (LEAF): e

Node 6 (+): children = Node1(a), Node2(b) → named: t6

Node 7 (+): children = Node4(d), Node5(e) → named: t8

Node 8 (-): children = Node6(t6), Node3(c) → named: t5

Node 9 (*): children = Node8(t5), Node7(t8) → named: t4

Node 10(-): children = Node9(t4), Node5(e) → named: t3

Node 11(+): children = Node6(t6), Node9(t4) → named: t2 [t6 REUSED]

Node 12(*): children = Node11(t2), Node10(t3) → named: t1

CSE detected by DAG

Node 6 ($a+b$) is shared by t_6 , and reused in Node 11 (t_6+t_4). Node 9 (t_4) is also reused in Node 11. This means $a+b$ need not be recomputed — a classic common sub-expression elimination.

Legal Evaluation Orders (Topological sorts of the DAG)

A legal evaluation order is any topological ordering of the DAG nodes. Two valid orders are:

Order 1: $t_6 \rightarrow t_8 \rightarrow t_5 \rightarrow t_4 \rightarrow t_3 \rightarrow t_2 \rightarrow t_1$

Order 2: $t_8 \rightarrow t_6 \rightarrow t_5 \rightarrow t_4 \rightarrow t_2 \rightarrow t_3 \rightarrow t_1$

(Any order that computes each operand before its operator is valid.)

Q2. Explain loop invariant computation with an example.

A computation inside a loop is called **loop-invariant** if its value does not change from one iteration of the loop to the next — i.e., all operands of the statement are either constants or have all their definitions outside the loop.

The optimization technique is called **Code Motion (Loop-Invariant Code Motion, LICM)**: we move loop-invariant statements to a newly created **pre-header** block that executes just once before the loop.

Conditions for Code Motion

- The statement s dominates all loop exits where the variable defined by s is live.
- There is only one definition of the variable defined by s in the loop.
- The variable defined by s is not live after the pre-header.

Example

```
/* Before optimization */
for (i = 0; i < n; i++) {
    x = a * b;           // loop-invariant: a, b not changed in loop
    arr[i] = x + i;
}

/* After Code Motion */
x = a * b;              // moved to pre-header
for (i = 0; i < n; i++) {
    arr[i] = x + i;
}
```

The multiplication $a * b$ was computed n times; after LICM it is computed once. This reduces runtime cost from $O(n)$ multiplications to $O(1)$.

Q3. What does code optimization mean in a compiler?

Code optimization is a phase in the compiler (between intermediate code generation and code generation) that transforms the intermediate representation of a program to produce code that:

- Runs faster (time optimization).
- Occupies less memory (space optimization).
- Consumes less energy / uses fewer resources.

Crucially, *the meaning (semantics) of the program must not change* — optimization is meaning-preserving transformation.

The optimization phase may operate at multiple levels: on basic blocks, on loops, globally across the entire procedure, or at the machine-code level.

Q4. Why is code optimization in a compiler required?

- **Performance:** High-level language constructs often produce naive intermediate code. Optimization removes redundant computations, making programs faster.
- **Memory efficiency:** Eliminates unnecessary temporaries and dead variables, reducing memory footprint.
- **Portable high-level code:** Programmers write clear, maintainable code; the compiler handles low-level efficiency.
- **Energy efficiency:** Fewer instructions = less CPU time = less battery/power consumed (critical for embedded and mobile systems).
- **Hardware utilization:** Machine-dependent optimizations exploit registers, cache, pipelining, and SIMD instructions that programmers rarely hand-tune.
- **Bridging the gap:** Compilers for languages like Python, Java, C++, etc. must compete with hand-written assembly — optimization closes this gap.

Q5. Define Machine Dependent and Machine Independent Code Optimization.

Machine Independent Code Optimization

Optimizations that can be applied to the intermediate code *without knowledge of the target machine's architecture*. They rely only on the program's logic.

Technique	Description
Common Sub-expression Elimination	Reuse already computed values (via DAG).
Copy Propagation	Replace use of copy 'x=y' with 'y' directly.
Dead Code Elimination	Remove code whose results are never used.
Constant Folding	Evaluate constant expressions at compile time.
Loop-Invariant Code Motion	Move loop-invariant code to pre-header.
Induction Variable Elimination	Simplify variables that change by fixed amounts each iteration.
Strength Reduction	Replace expensive operations with cheaper equivalents.

Machine Dependent Code Optimization

Optimizations applied during or after code generation that *depend on the specific features of the target hardware*.

Technique	Description
Register Allocation	Assign variables to registers to minimize memory accesses.
Instruction Selection	Choose the most efficient machine instructions.
Peephole Optimization	Replace short instruction sequences with faster ones.
Instruction Scheduling	Reorder instructions to avoid pipeline stalls.
Memory Hierarchy Optimization	Exploit cache locality (loop tiling, prefetching).

Q6. Algorithm for computing IN and OUT sets; GEN/KILL sets for a control flow graph; two iterations of IN/OUT.

Data-Flow Analysis — Background

Live Variable Analysis (a backward data-flow problem) tracks which variables are *live* (will be used later) at each program point. For each basic block B:

- **GEN[B]** — variables used in B before any definition in B (upward-exposed uses).
- **KILL[B]** — variables defined in B (killed for paths entering B from outside).
- **IN[B]** — variables live at the entry of B.
- **OUT[B]** — variables live at the exit of B.

Algorithm for Computing IN and OUT

```
/* Initialization */
for each basic block B:
    IN[B] = {}
    OUT[B] = {}

/* Iterative computation (backward) */
repeat
    for each basic block B (in reverse topological order):
        OUT[B] = UNION of IN[S] for all successors S of B
        IN[B] = GEN[B] UNION (OUT[B] - KILL[B])
until no IN or OUT set changes // fixed-point reached
```

How to Compute GEN and KILL for a Basic Block

Scan the block's instructions from **top to bottom**:

- For each instruction 'x = y op z': add y and z to GEN[B] if not already defined in B; add x to KILL[B].
- More precisely: GEN[B] = {variables with upward-exposed uses}; KILL[B] = {all variables defined in B}.

Note regarding Fig 1 (control flow graph)

Since Fig 1 was not included in the uploaded document, the specific GEN/KILL/IN/OUT values cannot be computed. Apply the algorithm above to whatever blocks appear in Fig 1: first compute GEN[B] and KILL[B] by scanning each block, then iterate the IN/OUT equations until convergence (typically 2–3 iterations for small CFGs).

Q7. Explain four types of structure-preserving transformations on basic blocks with examples.

1. Common Sub-expression Elimination (CSE)

If the same expression is computed more than once, and its operands haven't changed in between, replace subsequent computations with the already-computed value.

```
Before:  t1 = a + b
         t2 = a + b    // same expression
After:   t1 = a + b
         t2 = t1       // reuse t1
```

2. Dead Code Elimination

Remove assignments to variables that are never subsequently used. This does not change the observable output of the program.

```
Before:  x = 5          // x never used after this
         y = a + b
         return y
```

```
After:  y = a + b      // x = 5 removed
        return y
```

3. Renaming of Temporary Variables

Temporary variables in a basic block can be renamed to simplify analysis and enable further optimizations (e.g., converting to SSA form). This is structure-preserving because it only changes internal names.

```
Before:  t = a + b
        t = t * c      // t redefined
After:   t1 = a + b
        t2 = t1 * c    // separate names, easier to analyze
```

4. Interchange of Statements

Two adjacent statements can be swapped if they are independent (neither uses the output of the other, and they don't write to the same variable). This enables better instruction scheduling.

```
Before:  t1 = a + b
        t2 = c + d      // t2 does not depend on t1
After:   t2 = c + d      // swapped – semantically equivalent
        t1 = a + b
```

Q9. Basic blocks in compiler design; algorithm for partitioning three-address code; application to Fig 2.

Concept of Basic Blocks

A **basic block** is a maximal sequence of consecutive three-address instructions such that control enters the block only at the first instruction and leaves only at (or after) the last instruction — there are no jumps into the middle and no jumps out of the middle.

Basic blocks form the nodes of a **Control Flow Graph (CFG)**, where edges represent possible flow of control (sequential or via jumps).

Algorithm for Partitioning into Basic Blocks

Step 1 – Find leaders:

- (a) The first instruction of the program is a leader.
- (b) Any instruction that is the target of a conditional/unconditional jump is a leader.
- (c) Any instruction immediately following a conditional/unconditional jump is a leader.

Step 2 – Form basic blocks:

For each leader, its basic block consists of the leader itself plus all instructions up to (but not including) the next leader or end of program.

Application to Fig 2

Since Fig 2 was not included in the uploaded document, the specific blocks cannot be determined. Apply the above algorithm: identify leaders by scanning for the first instruction, jump targets, and fall-through instructions after jumps. Each leader starts a new basic block ending just before the next leader.

Q10. Explain strength reduction in compiler optimization.

Strength Reduction is an optimization that replaces an expensive (high-cost) operation with a cheaper equivalent one that produces the same result. It is commonly applied to **induction variables** inside loops.

Common Strength Reductions

Expensive Operation	Cheap Replacement	Condition
$i * 4$	$i \ll 2$ (left shift)	Multiply by power of 2
$i / 4$	$i \gg 2$ (right shift)	Divide by power of 2 (unsigned)
$i * i$ (in loop)	Running sum: $s += 2i+1$	Quadratic $\rightarrow$ linear
$a[*n+j]$ address	Incremental pointer update	Linear address in loop
$x ** 2$	$x * x$	Integer power of 2

Loop Example

```
/* Before strength reduction */
for (i = 0; i < n; i++)
    a[i] = i * 4;    // multiplication every iteration

/* After strength reduction */
t = 0;
for (i = 0; i < n; i++) {
    a[i] = t;
    t = t + 4;        // cheap addition replaces multiply
}
```

Q11. Write the algorithm for live variable analysis.

Live Variable Analysis is a *backward* data-flow analysis. A variable v is **live** at a point p if there exists a path from p to a use of v along which v is not redefined.

Definitions

- **USE[B]**: variables used in B before being defined in B (upward-exposed uses).
- **DEF[B]**: variables defined in B before any use of them in B.
- **IN[B]**: set of variables live at the entry of B.
- **OUT[B]**: set of variables live at the exit of B.

Data-Flow Equations

$OUT[B] = \text{UNION} \{ IN[S] : S \text{ is a successor of } B \}$

$IN[B] = \text{USE}[B] \text{ UNION } (OUT[B] - \text{DEF}[B])$

Algorithm

```
/* Step 1: Initialize */
for each basic block B:
    IN[B] = {}    // empty set
    OUT[B] = {}

/* Step 2: Compute USE[B] and DEF[B] for each block */
for each block B:
    scan instructions in B from top to bottom:
        for each instruction 'x = y op z':
            if y not in DEF[B]: add y to USE[B]
            if z not in DEF[B]: add z to USE[B]
```

```

        add x to DEF[B]

/* Step 3: Iterative fixed-point (process in REVERSE post-order) */
changed = true
while changed:
    changed = false
    for each basic block B (in reverse topological order):
        OUT_new = UNION of IN[S] for all successors S
        IN_new  = USE[B] UNION (OUT_new - DEF[B])
        if IN_new != IN[B] or OUT_new != OUT[B]:
            IN[B]  = IN_new
            OUT[B] = OUT_new
    changed = true

```

The algorithm terminates because the sets only grow (monotone) and are bounded by the finite set of all variables.

Q12. Write the algorithm for available expression in compiler optimization.

An expression **x op y** is **available** at a program point p if (1) every path from the entry to p evaluates x op y, and (2) after the last evaluation of x op y on each such path, neither x nor y is redefined. Available expression analysis is a *forward* data-flow analysis used for **Common Sub-expression Elimination (CSE)**.

Definitions

- **GEN[B]**: expressions computed in B that are available at the exit of B (i.e., computed in B and their operands not subsequently killed in B).
- **KILL[B]**: expressions that contain a variable defined in B (killed by assignments in B).
- **IN[B]**: expressions available at the entry of B.
- **OUT[B]**: expressions available at the exit of B.

Data-Flow Equations

```

IN[entry] = {}
IN[B]      = INTERSECTION { OUT[P] : P is a predecessor of B }   (for B != entry)
OUT[B]     = GEN[B] UNION (IN[B] - KILL[B])

```

Algorithm

```

/* Step 1: Compute GEN[B] and KILL[B] */
Let E = set of all expressions in the program.
for each block B:
    GEN[B] = {}
    KILL[B] = {}
    scan instructions in B from top to bottom:
        for each instruction 'x = y op z':
            // Add expression to GEN if operands not yet killed
            if y not killed in B so far AND z not killed in B so far:
                add (y op z) to GEN[B]
            // Kill all expressions involving x
            KILL[B] = KILL[B] UNION {all expressions in E that reference x}

```

```

        // Also remove x-involving expressions from GEN[B]
        remove from GEN[B] all expressions referencing x

/* Step 2: Initialize */
OUT[entry] = GEN[entry]
for each block B != entry:
    OUT[B] = E    // universal set (optimistic initialization)

/* Step 3: Iterative fixed-point (forward pass) */
changed = true
while changed:
    changed = false
    for each block B != entry (in topological order):
        IN_new = INTERSECTION of OUT[P] for all predecessors P
        OUT_new = GEN[B] UNION (IN_new - KILL[B])
        if OUT_new != OUT[B]:
            OUT[B] = OUT_new
            changed = true

```

Initializing $OUT[B] = E$ (the universal set of all expressions) gives the *maximum fixed point*, which is safe for CSE — we only mark an expression available if it is available on *all* paths (intersection).

Note: Questions referencing Fig 1 (CFG) and Fig 2 (three-address code) could not have their specific numerical answers computed because these figures were not included in the uploaded document. The complete algorithms required to solve them are provided above.